

Testing Pruebas de Software EV04

**Integrantes: Jorge Mendoza
Yeremi Navarro
Renato Escarate**

Asignatura: Testing pruebas de software

Docente: Marcelo Lara

Introducción:

En el siguiente trabajo, se presentará todo el conocimiento aprendido en la asignatura Testing pruebas de software, eso quiere decir que se mostrará documentación sobre casos de uso, casos de prueba, pruebas automatizadas en cypress, testing de apis y también pruebas de estrés con k6.

A continuación queda usted invitado a revisar nuestra documentación oficial.

Casos de uso:

En este apartado daremos a conocer los casos de uso que pudimos recolectar a partir de la página <https://cabanapinoahuacho.mlarac.cl/> y también de esta página <https://cabanapinoahuacho.mlarac.cl/admin/login>.

En este caso de uso, se establece como debería ser el uso del Login.

ID	1
Nombre	Login
Autor	Jorge
Descripción	Inicio de sesion
Actor	Usuario Administrador
Precondicione s	El usuario debe tener privilegios de administrador.
Flujo Normal	1.- Ir a la URL 2.- Ingresar User and password 3.- Hacer submit y llegar al dashboard
Flujo Alterno	1.- Ingresar user y password incorrecto
Excepciones	Mensaje de error de credenciales
Postcondicion es	El usuario administrador debe ingresar y ver el dashboard

En este caso se establece cómo se cancela una reserva desde el panel de administración de la página web.

ID	2
Nombre	Cancelar reserva
Autor	yeremi
Descripción	cancelar reserva
Actor	Usuario Administrador
Precondiciones	El usuario debe haber ingresado. El usuario debe tener privilegios de administrador. El usuario debe tener privilegio de administrador.
Flujo Normal	1.- Ir a la URL 2.- Ingresar User and password 3.- Hacer submit y llegar al dashboard, click en gestionar reservas, click boton en cancelar
Flujo Alternativo	
Excepciones	
Postcondiciones	El administrador puede ver y cancelar reservas

En este caso de uso, se ve cómo se gestiona un cambio de precio exitoso.

ID	3
Nombre	Gestion de precios
Autor	Jorge
Descripción	Cambiar precios segun categoría
Actor	Usuario Administrador
Precondiciones	El usuario debe haber ingresado. El usuario debe tener privilegios de administrador el usuario debe tener acceso al dashboard
Flujo Normal	Ir a la opción Gestion de precios, elegir categoría de precio y ingresar un valor
Flujo Alternativo	
Excepciones	
Postcondiciones	Debe salir mensaje precios por categoría actualizados correctamente

En este caso se ve reflejado una reserva con campos vacíos.

ID	4
Nombre	campos vacios
Autor	yeremi
Descripción	verificar que el sistema no permita procesar una reserva sin rellenar los campos solicitados
Actor	Usuario
Precondicione s	El usuario debe haber ingresado a la pagina de reservas
Flujo Normal	1.- Ir a la URL 2.- click en boton reservar ahora, dejar campos vacíos, click en reservar
Flujo Alternativo	
Excepciones	
Postcondicion es	El sistema no permite avanzar en el proceso de reserva y muestra el mensaje de advertencia: "porfavor completa todos los campos de fecha".

En este apartado se prueba la funcionalidad de cambiar precio.

ID	5
Nombre	
Autor	Jorge
Descripción	Cambiar precios por rango de fechas
Actor	Usuario administrador
Precondicione s	El usuario debe tener acceso al dashboard haber iniciado sesiones
Flujo Normal	Ingreso de fecha de inicio, Ingreso de fecha de salida, ingresar nuevo valor, cambiar categoria de precio
Flujo Alternativo	
Excepciones	
Postcondicion es	Debe salir mensaje que los precios fueron actualizados correctamente

Apartado de edición de precio en el dashboard

ID	6
Nombre	
Autor	Jorge
Descripción	Editar precio en el dashboard
Actor	Usuario administrador
Precondicione s	El usuario debe tener acceso al dashboard haber iniciado sesion
Flujo Normal	Ir a gestión de precios, luego hacer click en la parte de abajo donde dice editar, última fila, modificar precio y la categoría
Flujo Alternativo	
Excepciones	
Postcondicion es	Precio individual modificado correctamente

Reserva exitosa (happy path)

ID	7
Nombre	Reserva exitosa
Autor	Jorge
Descripción	
Actor	Usuario
Precondicione s	El usuario debe tener acceso a la página web
Flujo Normal	1.- Ir a la URL 2.- Click en reservar 3.- Llenar los campos 4.- Click en el boton 5.- Vista del detalle
Flujo Alternativo	
Excepciones	
Postcondicion es	Reserva gestionada

A partir de los casos de uso, seguiremos a los casos de prueba.

Casos de prueba:

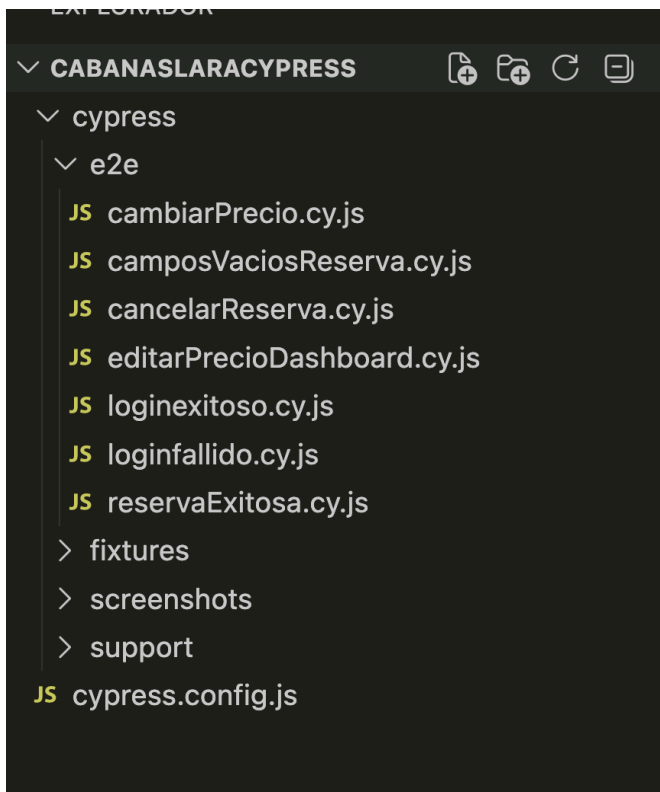
Para esta parte del informe, se establece una matriz de casos de prueba, se componen de filas y columnas las cuales detallan información de las pruebas realizadas.

Id	Nombre	Descripción	Precondiciones	Pasos de Ejecución	Datos de entrada	Resultado Esperado	Estado (pasó/falló)
CASO-01	Login exitoso	Ingreso exitoso de usuario y contraseña	El usuario debe estar creado y tener permisos de administración	Ingreso a la URL, luego ingresar usuario y contraseña en los campos y hacer click en ingresar	usuario: admin contraseña: admin123	Debe ingresar y ver el dashboard	pasó
CASO-02	Login incorrecto	Ingreso incorrecto de usuario o contraseña	El usuario debe estar creado y tener permisos de administración	Ingreso a la URL, luego ingresar usuario y contraseña en los campos y hacer click en ingresar	usuario: administrador contraseña: admin1234	Mensaje de error y volver a ingresar los datos	pasó
CASO-03	Cancelar reserva	Se realiza reserva cancelada de forma exitosa	El usuario debe estar creado y tener acceso a la web y debe existir una reserva	En el dashboard, click en reservas y después en la tabla hacer click en cancelar	Null	Mensaje ¿Estás seguro de que deseas borrar la reserva? hacer click en si	pasó
CASO-04	Cambiar precio	Se realiza cambio de precio según categoría y se ingresa un valor	El usuario debe estar creado y tener acceso a la web	Ir a la opción Gestión de precios, elegir categoría de precio y ingresar un valor	Numero entero	Precio modificado correctamente	pasó

CASO-05	Campos Vacios Reserva	Se realiza reserva con campos vacios	Tener acceso a las reservas	1.- Ir a la URL 2.- click en boton reservar ahora, dejar campos vacíos, click en reservar	Null	Mensaje de error , Por favor completa todos los campos de fechas	pasó
CASO-06	Editar precio Dashboard	Editar precio en el dashboard	El usuario debe estar creado y tener acceso a la web	Ir a gestión de precios, luego hacer click en la parte de abajo donde dice editar, última fila, modificar precio.	Numero entero	Precio individual modificado o exitosamente	pasó
CASO-07	Reserva exitosa	En este apartado se hace una reserva exitosa	tener acceso funcional a la página	1.- Ir a la URL 2.- Click en reservar 3.- Llenar los campos 4.- Click en el boton 5.- Vista del detalle	Nombre, correo, ocupantes, fecha	Página final donde se aprecia mensaje reserva exitosa	pasó

De nuestra matriz de casos de prueba, hemos podido realizar las pruebas cypress.

En este apartado, se analizan 7 casos de pruebas sacados de los casos de uso documentados anteriormente, cada prueba está automatizada y mediante código Javascript, se puede ir probando funciones de la página de forma más rápida, los beneficios de este programa, hace que puedas ser más productivo a la hora de realizar testing,



Adjuntamos el código de cada prueba.

Cambiar precio

En este caso, se describe cypress, una serie de instrucciones que facilitan el testing, este sería el cambiar precio donde la “alert” final debe ser visible, significando que la prueba fue un éxito,

```
1 describe('cambiar precio', () => {
2   it('passes', () => {
3     cy.visit('https://cabanapinohuacho.mlarac.cl/admin/login')
4     cy.get('#username').type('admin')
5     cy.get('#password').type('admin123')
6     cy.get('.btn').click()
7     cy.get('.list-group > [href="/admin/precios"]').click()
8     cy.get('#categoryPrice').type('50000')
9     cy.get('#updateCategoryForm > .btn').click()
10
11    cy.get('.alert', { timeout: 3000 })
12      .should('be.visible')
13      .invoke('text')
14      .then((text) => {
15        cy.log('Texto real del alert: ' + text)
16      })
17    })
18  })
```

Reserva con datos (happy path), las variables del día de llegada y salida deben ir cambiando para hacer de la prueba de mejor forma.

```
describe('reservar', () => {
  it('permite realizar una reserva exitosa', () => {
    cy.visit('https://cabanapinoahuacho.mlarac.cl/')
    cy.get('.hero-buttons a[href="/reservar"]').click()
    cy.url().should('include', '/reservar')
    cy.get('[name="checkIn"]').type('2026-07-30')
    cy.get('[name="checkOut"]').type('2026-08-30')
    cy.get('[name="guests"]').select('1')
    cy.get('.text-center > .btn').first().click()
    cy.get('[name="guestName"]').type('Renato Escarate')
    cy.get('[name="guestEmail"]').type('renato.esc@ejemplo.com')
    cy.get('[name="guestPhone"]').type('+56996576890')
    cy.get('.d-flex > .btn-primary').click()
    cy.get('#bookingForm').submit()
    cy.get('.display-5').should('be.visible').and('contain', '¡Reserva exitosa!')

  })
})
```

Reservar sin datos

Como no hay datos en los inputs, se validan con un should y debe contener “Por favor completa todos los campos de fechas”, evidenciando la funcionalidad de la web.

```
1 describe('reservar sin datos', () => {
2   it('no permite continuar sin ingresar datos', () => {
3     cy.visit('https://cabanapinoahuacho.mlarac.cl/')
4     cy.get('.hero-buttons a[href="/reservar"]').click()
5     cy.url().should('include', '/reservar')
6
7     cy.get('div.text-center > .btn').first().click()
8
9     cy.get('.alert-warning')
10      .should('be.visible')
11      .and('contain', 'Por favor completa todos los campos de fechas')
12   })
13 })
```

Cancelar reserva:

En esta captura, se expresa el flujo de entrada al dashboard, luego ir al apartado reservar para luego hacer click en los botones cancelar la reserva y obviamente la alert debe ser visible y contener el texto “cancelada”

```
1  describe('cancelar_reserva', () => {
2    it('passes', () => {
3      cy.visit('https://cabanapinoahuacho.mlarac.cl/admin/login')
4      cy.get('#username').type('admin')
5      cy.get('#password').type('admin123')
6      cy.get('.btn').click()
7      cy.get('.list-group > [href="/admin/reservas"]').click()
8
9      // Acepta automáticamente el confirm() nativo del navegador
10     cy.on('window:confirm', () => true)
11
12     // Toma la reserva más reciente que aún tenga botón de cancelar disponible
13     cy.get('[id^="reservation-"]')
14       .filter(':has(.text-center > .btn)')
15       .first()
16       .find('.text-center > .btn')
17       .click()
18
19     // Verifica que la cancelación fue exitosa
20     cy.get('.alert', { timeout: 10000 })
21       .should('be.visible')
22       .and('contain', 'cancelada')
23   })
24 })
```

Editar_precio_dashboard:

En este apartado damos instrucciones para editar un precio en el dashboard.

```
1 describe('editar_precio_dashboard', () => {
2   it('passes', () => {
3     cy.visit('https://cabanapinoahuacho.mlarac.cl/admin/login')
4     cy.get('#username').type('admin')
5     cy.get('#password').type('admin123')
6     cy.get('.btn').click()
7     cy.get('.list-group > [href="/admin/precios"]').click()
8     cy.get(':nth-child(1) > :nth-child(6) > .btn').click()
9     cy.get(':nth-child(3) > .input-group > [name="price"]').click().clear().type('50000')
10    cy.get('.modal-footer > .btn-primary').click()
11
12    cy.get('.alert-success', { timeout: 5000 })
13      .should('be.visible')
14      .invoke('text')
15      .then((text) => {
16        cy.log('Texto de la alerta: ' + text)
17      })
18  })
19 })
```

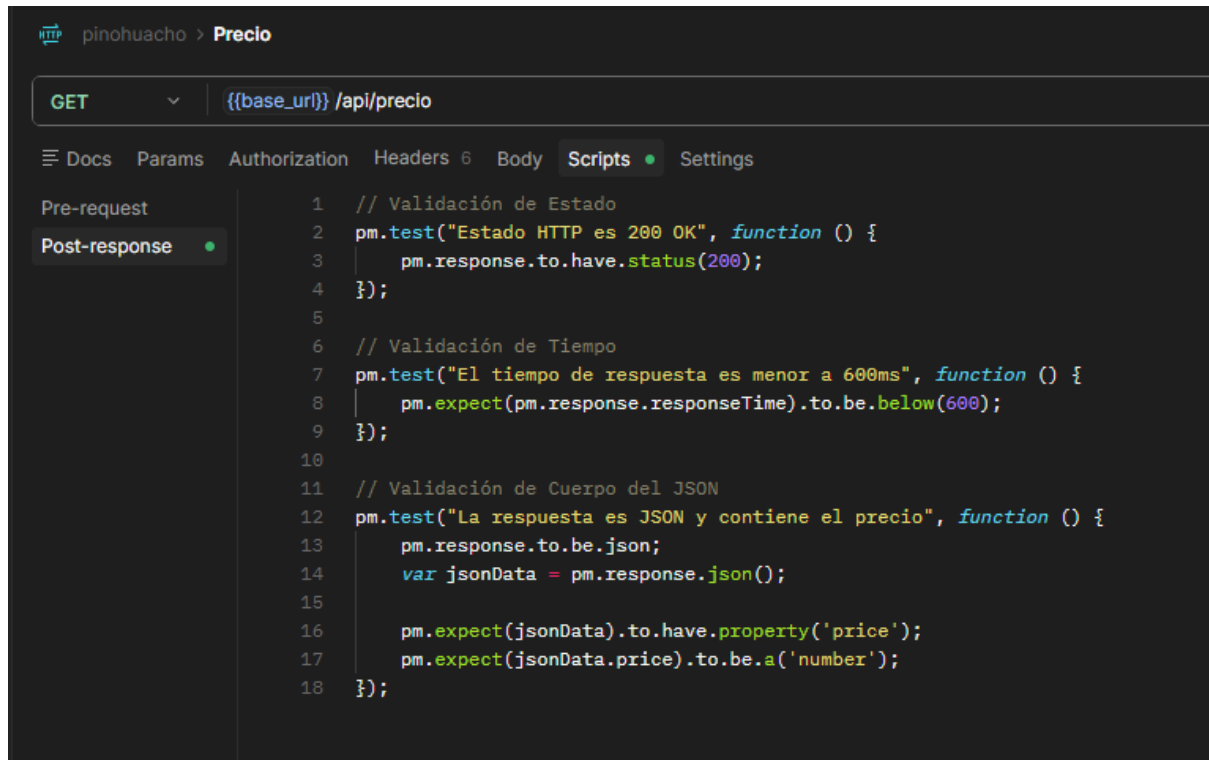
login_exitoso: busca conseguir un Login exitoso para ingresar a la plataforma.

login_fallido:

```
1 describe('login_fallido', () => {
2   it('passes', () => {
3     cy.visit('https://cabanapinoahuacho.mlarac.cl/admin/login')
4
5     cy.get('#username').type('admin')
6     cy.get('#password').type('wrongpassword')
7     cy.get('.btn').click()
8
9     // Verifica que el mensaje de error sea visible
10    cy.get('li').should('be.visible').and('contain', 'Credenciales inválidas')
11
12    // Verifica que NO redirigió al panel de admin
13    cy.url().should('include', '/login')
14
15    // Verifica que sigue en la pantalla de login (el form de login sigue visible)
16    cy.get('#username').should('be.visible')
17    cy.get('#password').should('be.visible')
18  })
19 })
```

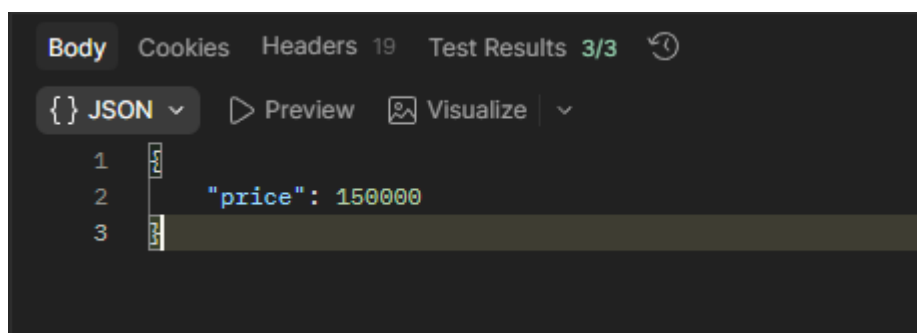
Documentación de la API.

Prueba de precio: Esta prueba busca un get para conocer el valor de una reserva esperando una respuesta en formato JSON en menos de 600ms.



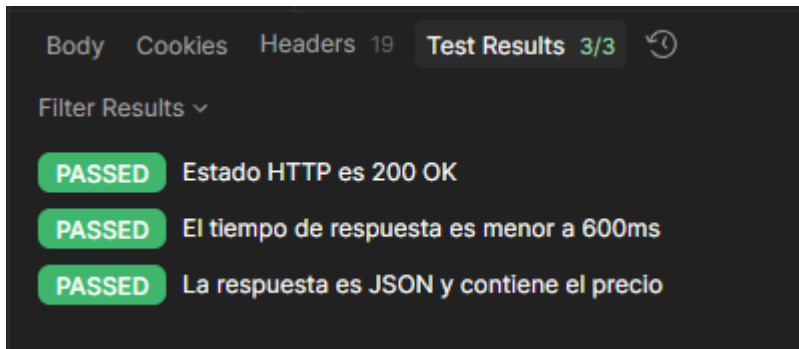
```
1 // Validación de Estado
2 pm.test("Estado HTTP es 200 OK", function () {
3   pm.response.to.have.status(200);
4 });
5
6 // Validación de Tiempo
7 pm.test("El tiempo de respuesta es menor a 600ms", function () {
8   pm.expect(pm.response.responseTime).to.be.below(600);
9 });
10
11 // Validación de Cuerpo del JSON
12 pm.test("La respuesta es JSON y contiene el precio", function () {
13   pm.response.to.be.json;
14   var jsonData = pm.response.json();
15
16   pm.expect(jsonData).to.have.property('price');
17   pm.expect(jsonData.price).to.be.a('number');
18 });
```

Resultado:



```
1 {}
2 {"price": 150000}
3 {}
```

Resultado de test:



Consulta disponibilidad: Esta prueba busca conseguir un listado de disponibilidad de cabañas por una fecha definida, esperando que devuelva un JSON que no es un arreglo, en menos de 600ms y que el valor esté entre occupied o available.

```
GET {{base_url}} /api/availability/2026/07

Docs Params Authorization Headers 6 Body Scripts Settings

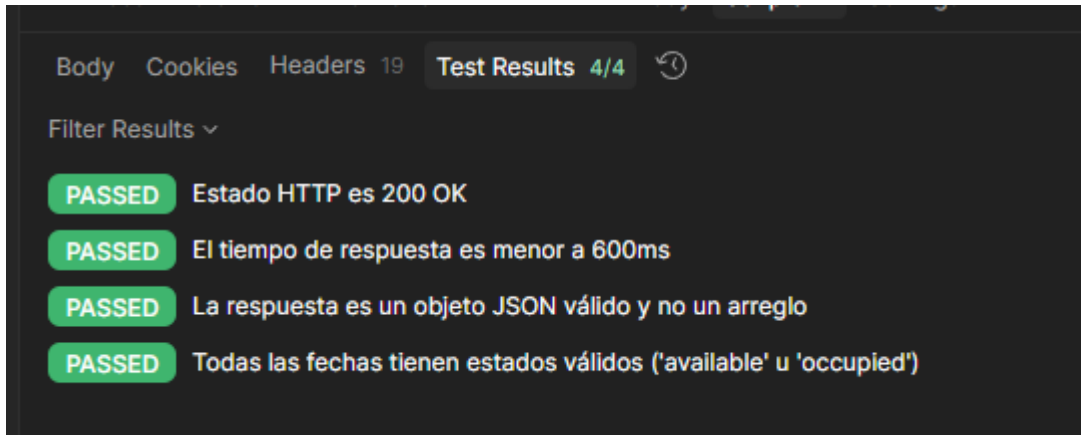
Pre-request
Post-response

1 // Validación de Estado
2 pm.test("Estado HTTP es 200 OK", function () {
3   pm.response.to.have.status(200);
4 });
5
6 // Validación de Tiempo
7 pm.test("El tiempo de respuesta es menor a 600ms", function () {
8   pm.expect(pm.response.responseTime).to.be.below(600);
9 });
10
11 // Validación de la estructura del objeto JSON
12 pm.test("La respuesta es un objeto JSON válido y no un arreglo", function () {
13   pm.response.to.be.json;
14   var jsonData = pm.response.json();
15
16   // Validamos que sea un objeto {} y no un array []
17   pm.expect(jsonData).to.be.an('object');
18   pm.expect(Array.isArray(jsonData)).to.be.false;
19 });
20
21 // Validación robusta de los datos internos
22 pm.test("Todas las fechas tienen estados válidos ('available' u 'occupied')", function () {
23   var jsonData = pm.response.json();
24
25   // Extraemos todas las fechas (las llaves del objeto)
26   var fechas = Object.keys(jsonData);
27
28   // Verificamos que el mes traiga al menos un día (que no esté vacío)
29   pm.expect(fechas.length).to.be.above(0);
30
31   // Recorremos cada fecha para validar su contenido
32   fechas.forEach(function(fecha) {
33     let estado = jsonData[fecha];
34
35     // Validamos que el estado de cada día sea estrictamente una de las dos opciones esperadas
36     pm.expect(estado).to.be.oneOf(['available', 'occupied']);
37   });
38 });
```

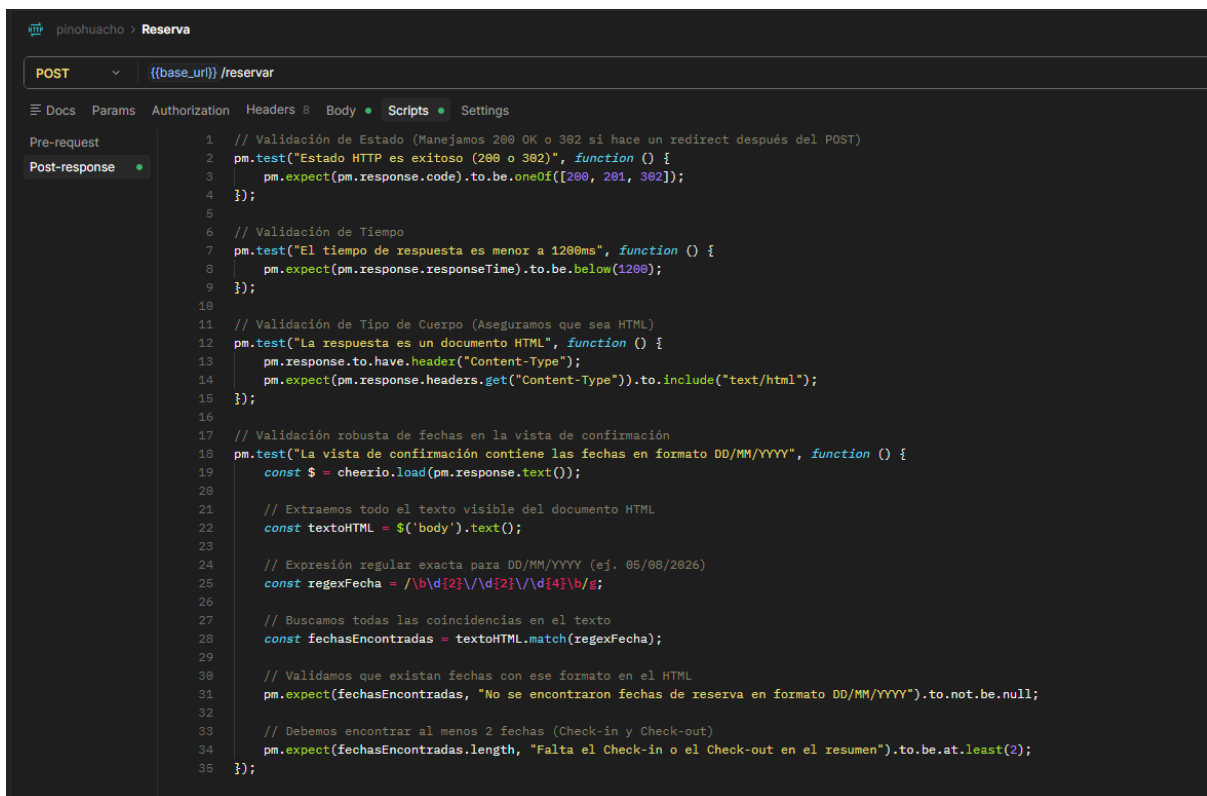
Respuesta:

```
Body Cookies Headers 19 Test Results 4/4 ↺
{ } JSON Preview Visualize
1 {
2   "2026-07-01": "occupied",
3   "2026-07-02": "occupied",
4   "2026-07-03": "occupied",
5   "2026-07-04": "occupied",
6   "2026-07-05": "occupied",
7   "2026-07-06": "occupied",
8   "2026-07-07": "occupied",
9   "2026-07-08": "occupied",
10  "2026-07-09": "available",
11  "2026-07-10": "occupied",
12  "2026-07-11": "occupied",
13  "2026-07-12": "occupied",
14  "2026-07-13": "available",
15  "2026-07-14": "available",
16  "2026-07-15": "occupied",
17  "2026-07-16": "occupied",
18  "2026-07-17": "occupied",
19  "2026-07-18": "available",
20  "2026-07-19": "available",
21  "2026-07-20": "occupied",
22  "2026-07-21": "occupied",
23  "2026-07-22": "occupied",
24  "2026-07-23": "occupied",
25  "2026-07-24": "occupied",
26  "2026-07-25": "available",
27  "2026-07-26": "available",
28  "2026-07-27": "available",
29  "2026-07-28": "available",
30  "2026-07-29": "available",
31  "2026-07-30": "available",
32  "2026-07-31": "occupied"
33 }
```

Resultado de test:



Prueba de reserva: Esta prueba busca hacer un post en el módulo de reservas de cabañas, esperando un documento de confirmación HTML. Sin embargo no pasó una validación que tiene que ver con el formato de fecha esperado.



Respuesta HTML:

```

Body Cookies Headers 20 Test Results 3/4
HTML Preview Visualize
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Reservar Cabaña - Pino Huacho</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
8   <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600;700&display=swap" rel="stylesheet">
9   <link rel="stylesheet" href="/css/styles.css">
10  <link rel="stylesheet" href="/css/booking.css">
11 </head>
12 <body>
13   <nav class="navbar navbar-expand-lg navbar-dark bg-primary fixed-top">
14     <div class="container">
15       <a class="navbar-brand" href="/">
16         <strong>Cabaña Pino Huacho</strong>
17       </a>
18       <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
19         <span class="navbar-toggler-icon"></span>
20       </button>
21
22       <div class="collapse navbar-collapse" id="navbarNav">
23         <ul class="navbar-nav ms-auto">
24           <li class="nav-item">
25             <a class="nav-link" href="/">Inicio</a>
26           </li>
27           <li class="nav-item">
28             <a class="nav-link" href="/caracteristicas">Características</a>
29           </li>
30           <li class="nav-item">
31             <a class="nav-link btn btn-warning text-dark px-3 mx-2" href="/reservar">Reservar Ahora</a>
32           </li>
33         </ul>
34       </div>
35     </div>
36   </nav>
37
38   <main>

```

Resultado de pruebas:

```

Body Cookies Headers 20 Test Results 3/4 200 OK 165 ms
Filter Results
PASSED Estado HTTP es exitoso (200 o 302)
PASSED El tiempo de respuesta es menor a 1200ms
PASSED La respuesta es un documento HTML
FAILED La vista de confirmación contiene las fechas en formato DD/MM/YYYY | AssertionError: No se encontraron fechas de reserva en formato DD/MM/YYYY: expected null not to be null

```

Código fuente k6: (pruebas de estrés)

En este apartado configuramos las pruebas, en este caso usaremos las pruebas realizadas en postman para poder hacer las pruebas de estrés con K6, simulando 50 usuarios virtuales y en la variable URL seteamos la página a testear cabañas Pinohuacho

```
import http from 'k6/http';
import { check, sleep } from 'k6';

// =====
// Fase 4: Pruebas de Carga y Estrés con k6
// =====

// Configuración de los escenarios
export const options = {
  stages: [
    { duration: '1m', target: 50 }, // Ramping up a 50 VUs
    { duration: '2m', target: 50 }, // Meseta de estabilización
    { duration: '1m', target: 0 }, // Ramping down a 0
  ],
  // Umbrales de éxito (globales)
  thresholds: {
    http_req_duration: ['p(90)<800', 'p(95)<1200'],
    http_req_failed: ['rate<0.01'],
    // Umbrales específicos por endpoint
    'http_req_duration{endpoint:precio}': ['p(95)<800'],
    'http_req_duration{endpoint:disponibilidad}': ['p(95)<800'],
    'http_req_duration{endpoint:reserva}': ['p(95)<1500'],
  },
};

const BASE_URL = 'https://cabanapinohuacho.mlarac.cl';

export default function () {
  // 1. Endpoint: Precio
  let resPrecio = http.get(`${BASE_URL}/api/precio`, {
    tags: { endpoint: 'precio' },
  });
  check(resPrecio, {
    'Precio - status es 200': (r) => r.status === 200,
  });
}
```

```
'Precio - contiene propiedad price': (r) => {
  try {
    return r.json().hasOwnProperty('price');
  } catch (e) {
    return false;
  }
},
});
sleep(1);

// 2. Endpoint: Disponibilidad
let resDispo = http.get(`${BASE_URL}/api/availability/2026/07`, {
  tags: { endpoint: 'disponibilidad' },
});
check(resDispo, {
  'Disponibilidad - status es 200': (r) => r.status === 200,
  'Disponibilidad - es un objeto': (r) => {
    try {
      const body = r.json();
      return typeof body === 'object' && !Array.isArray(body);
    } catch (e) {
      return false;
    }
  },
});
sleep(1);

// 3. Endpoint: Reserva (POST application/x-www-form-urlencoded)
const payloadReserva = {
  guestName: 'Renato Escárate',
  guestEmail: 'rescaratef@gmail.com',
  guestPhone: '+56990042992',
  checkIn: '2026-08-05',
  checkOut: '2026-08-10',
  guests: '2',
  notes: 'Llegaremos tarde por la noche',
};

const paramsReserva = {
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
  },
},
```

```
tags: { endpoint: 'reserva' },
};

let resReserva = http.post(
  `${BASE_URL}/reservar`,
  payloadReserva,
  paramsReserva
);
check(resReserva, {
  'Reserva - status es exitoso (200, 201 o 302)': (r) =>
    [200, 201, 302].includes(r.status),
  'Reserva - es HTML': (r) =>
    r.headers['Content-Type'] &&
    r.headers['Content-Type'].includes('text/html'),
});

sleep(2);
}

// =====
// Cómo ejecutarlo:
// k6 run load-test.js
//
// Para guardar el resumen en un archivo
// k6 run load-test.js --summary-export=resumen.json
//
```

Explicación.

Configuración del escenario

Prueba de carga con 3 etapas (ramping-vus) sobre 3 endpoints: precio (GET), disponibilidad (GET) y reserva (POST). Duración total: 4m30s. URL base: cabanapinoahuacho.mlarac.cl

Etapas	Duración	VUs objetivo	Descripción
1	1 min	0 → 50	Ramp-up
2	2 min	50	Meseta / carga sostenida
3	1 min	50 → 0	Ramp-down

Umbrales (thresholds) definidos

Métrica	Umbral	Alcance
http_req_duration	p90<800ms p95<1200ms	Global
http_req_failed	rate<1%	Global
{endpoint:precio}	p95<800ms	Específico
{endpoint:disponibilidad}	p95<800ms	Específico
{endpoint:reserva}	p95<1500ms	Específico

Checks funcionales

Precio: status 200 y existencia de propiedad price. Disponibilidad: status 200 y respuesta tipo objeto. Reserva: status 200/201/302 y respuesta HTML.

Resultados obtenidos

Checks y tiempos de respuesta

Métrica	Valor		
Checks totales	11,580 (100% éxito)		
Requests HTTP	5,790 (23.94 req/s)		
Requests fallidos	0 (0.00%)		
Iteraciones	1,930 (0 interrumpidas)		

Endpoint	avg	p90	p95	max
Global	243.94ms	391.57ms	572.49ms	1.5s
Precio	202.57ms	209.69ms	216.89ms	1.02s
Disponibilidad	211.67ms	222.35ms	235.32ms	1.5s
Reserva	317.58ms	581.79ms	716.45ms	1.49s

Cumplimiento de umbrales

Los 6 thresholds definidos se cumplieron en su totalidad: p90=391.57ms (<800ms), p95=572.49ms (<1200ms), precio p95=216.89ms, disponibilidad p95=235.32ms, reserva p95=716.45ms, y tasa de error=0.00% (<1%).

Evidencia de ejecución

```

TOTAL RESULTS

checks_total.....: 11580   47.8869/s
checks_succeeded...: 100.00% 11580 out of 11580
checks_failed.....: 0.00%   0 out of 11580

✓ Precio - status es 200
✓ Precio - contiene propiedad price
✓ Disponibilidad - status es 200
✓ Disponibilidad - es un objeto
✓ Reserva - status es exitoso (200, 201 o 302)
✓ Reserva - es HTML

HTTP
http_req_duration.....: avg=243.94ms min=157.25ms med=198.89ms max=1.5s p(90)=391.57ms p(95)=572.49ms
{ endpoint:disponibilidad }...: avg=211.67ms min=163.43ms med=201.73ms max=1.5s p(90)=222.35ms p(95)=235.32ms
{ endpoint:precio }.....: avg=202.57ms min=157.25ms med=192.04ms max=1.02s p(90)=209.69ms p(95)=216.89ms
{ endpoint:reserva }.....: avg=317.58ms min=159.18ms med=206.61ms max=1.49s p(90)=581.79ms p(95)=716.45ms
{ expected_response:true }...: avg=243.94ms min=157.25ms med=198.89ms max=1.5s p(90)=391.57ms p(95)=572.49ms
http_req_failed.....: 0.00% 0 out of 5790
http_reqs.....: 5790   23.94345/s

EXECUTION
iteration_duration.....: avg=4.74s min=4.52s med=4.62s max=7.96s p(90)=5.02s p(95)=5.25s
iterations.....: 1930   7.98115/s
vus.....: 1 min=1 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 28 MB 114 kB/s
data_sent.....: 799 kB 3.3 kB/s

running (4m01.8s), 00/50 VUs, 1930 complete and 0 interrupted iterations
default ✓ [=====] 00/50 VUs 4m0s
PS C:\Users\yo\Desktop\k6>
```

```
execution: local
script: script.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 4m30s max duration (incl. graceful stop):
  * default: Up to 50 looping VUs for 4m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

THRESHOLDS

```
http_req_duration
✓ 'p(90)<800' p(90)=391.57ms
✓ 'p(95)<1200' p(95)=572.49ms

{endpoint:disponibilidad}
✓ 'p(95)<800' p(95)=235.32ms

{endpoint:precio}
✓ 'p(95)<800' p(95)=216.89ms

{endpoint:reserva}
✓ 'p(95)<1500' p(95)=716.45ms

http_req_failed
✓ 'rate<0.01' rate=0.00%
```

Observaciones

- Reserva (POST) tiene el mayor tiempo de respuesta (p95=716.45ms) por ser operación de escritura, pero dentro del umbral.
- Precio y Disponibilidad responden con amplio margen bajo sus umbrales (p95<240ms).
- 0% de errores con hasta 50 VUs concurrentes, indicando estabilidad del backend.
 - Máquina usada windows 11, specs CPU: Ryzen 5 7600 5,1 ghz, 32 gb ram, SSD.

Conclusiones finales:

En este trabajo realizado pusimos en práctica lo que aprendimos en la asignatura de Testing de pruebas de software, para ello realizamos varios casos de prueba, casos de uso y pruebas automatizadas en el sitio web Cabañas Pino Huacho. Pudimos lograr verificar todas las pruebas de testing mencionadas, en base a esto podemos reflexionar que como futuros profesionales es necesario conocer el proceso de testing en el software ya que no existe el desarrollo perfecto del mismo, sacar un programa a producción no es solo programar algo, se tiene que documentar y hacer rigurosas pruebas de funcionamiento, de casos de uso y pruebas de estrés para cumplir niveles de calidad y seguridad del software para validar un proceso de producción.